

UML 2.5 CLASS MODEL GUIDE

# AeroBook Flight Platform

## Complete UML Class Diagram & Domain Model Specification

Detailed architectural breakdown of JPA Entities, Value Objects, Enumerations, Multiplicities, and Cross-Microservice Domain Mappings.

### DOMAIN MODELING PARAMETERS

ORM Framework:

Spring Data JPA & Hibernate 6

Total Persistent Entities:

9 Core Domain Entities

Visual Theme:

Black Canvas & White Marker

Notation Standard:

UML 2.5 Object Model

Relationship Types:

Composition, Aggregation & FK

Database Isolation:

Database-per-Service (MySQL 8)

### Executive Summary

In enterprise software engineering, the UML Class Diagram represents the static structure of a system by declaring its entities, attributes, operations, and relationships. In a microservices architecture, class diagrams possess a distinctive trait: entities are grouped into strict Bounded Contexts where each microservice exclusively owns its domain entities and persistence tables.

This engineering guide provides a complete, easy-to-understand reference for the AeroBook Class Diagram. It walks through the 9 core JPA entities (Credential, RefreshToken, UserEntity, Aircraft, Flight, Fare, Booking, Passenger, CheckIn), explaining their attributes, visibility modifiers (+ public, - private), methods, composition lifecycles, and logical cross-microservice references.

### DOCUMENT TABLE OF CONTENTS

Page 2

High-Resolution UML Class Diagram (Full Landscape Vector)

Complete static structure blueprint in black canvas & white marker theme

Page 3

Section 1: Fundamentals of UML Class Modeling

Compartments, attributes, visibility modifiers (-/+), and relationship types

Page 4

Section 2: Identity & Traveler Domain Model

auth-service (Credential, RefreshToken, Role) and user-service (UserEntity)

Page 5

Section 3: Flight Scheduling & Yield Pricing Domain Model

flight-service (Aircraft, Flight, FlightStatus) and fare-service (Fare)

Page 6

Section 4: Booking Orchestration & Check-In Domain Model

booking-service (Booking, Passenger, PnrGenerator) & check-in-service (CheckIn)

Page 7

Section 5: Cross-Cutting DTOs & Microservice Bounded Contexts

ErrorResponse, Jackson non-null serialization, and logical cross-service FKs

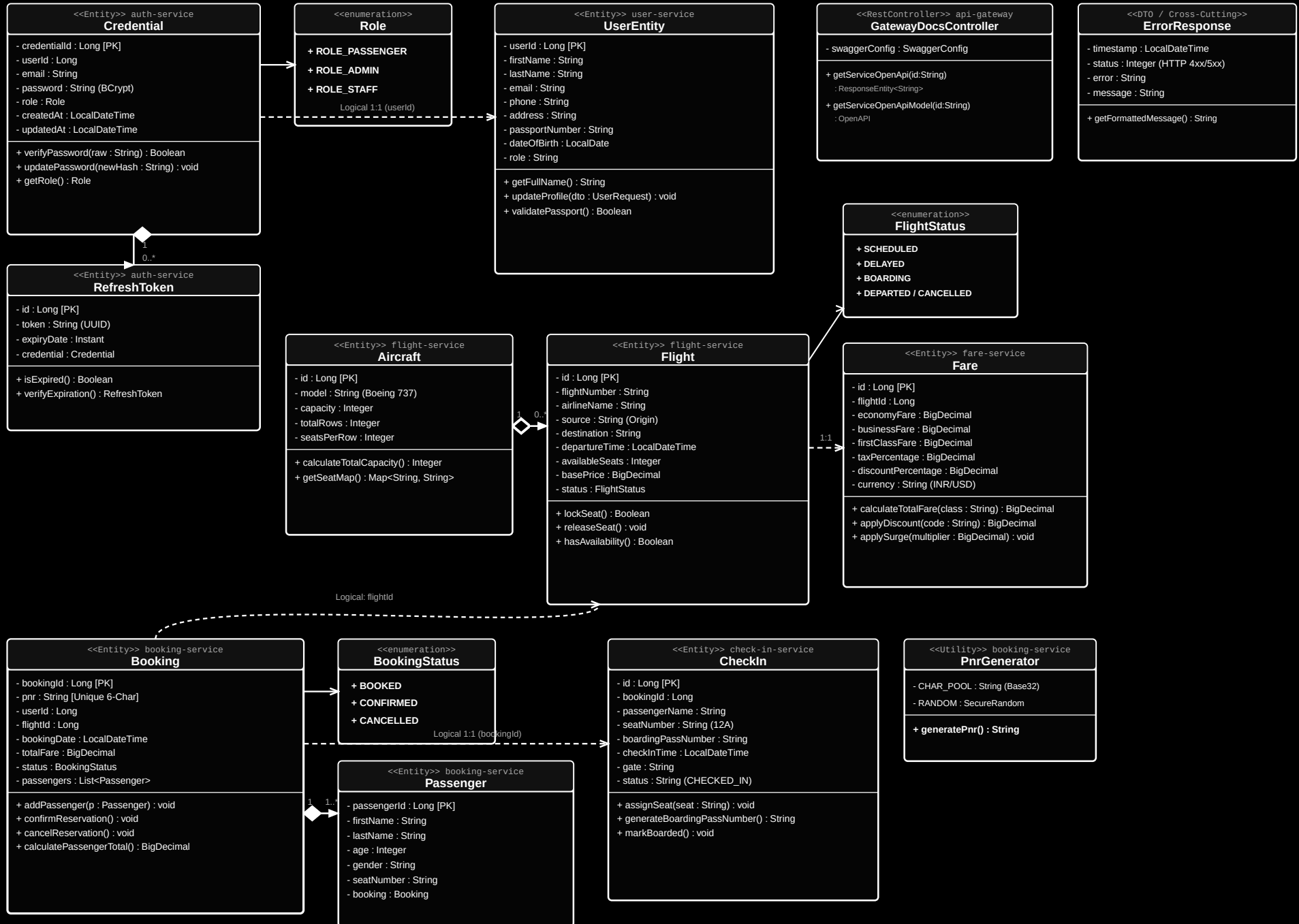
Page 8

Section 6: Complete Entity Traceability Matrix & Defense FAQ

Class-to-table dictionary and 4 key technical review defense questions & answers

# AEROBOOK PLATFORM - SYSTEM UML CLASS DIAGRAM

Enterprise Microservices Domain Model • JPA Entities, DTOs & Multiplicities • Black Canvas & White Marker Theme



# 1. Fundamentals: What is a UML Class Diagram?

UML 2.5 Object-Oriented Principles Explained in Plain English

A Class Diagram describes the internal structure of a software system by modeling its classes, their attributes (data fields), their operations (methods), and the relationships between them. In Java, each class box maps directly to a .java entity or model file.

## Top Compartment: Class Header & Stereotype

Declares the name of the class (e.g. Booking) and its architectural stereotype such as <<Entity>>, <<enumeration>>, <<RestController>>, or <<DTO>>.

## Middle Compartment: Attributes (State)

Lists instance variables with visibility prefixes (- for private, + for public), name, and data type (e.g. - pnr : String).

## Bottom Compartment: Operations (Behavior)

Declares public business methods, parameters, and return types (e.g. + calculateTotalFare(class : String) : BigDecimal).

# UML Relationship Types in AeroBook

How Classes Connect and Interact

## Composition (Filled Diamond \*--): Strong "Part-of" Ownership

Signifies strict lifecycle dependency. The child cannot exist without the parent. In AeroBook, Booking \*-- Passenger is a composition: if a booking reservation is deleted, all attached passenger records are automatically deleted by JPA (CascadeType.ALL, orphanRemoval = true).

## Aggregation (Hollow Diamond o--): Shared "Has-a" Association

Signifies that a child belongs to a parent, but has an independent lifecycle. In AeroBook, Aircraft o-- Flight is an aggregation: an aircraft operates many flights, but if a specific flight is cancelled, the physical aircraft continues to exist.

## Logical Cross-Service Reference (Dashed Line ..>): Database Isolation

In a microservice architecture, foreign key constraints CANNOT cross physical database schemas. Therefore, Booking references Flight via a logical flightId number, and CheckIn references Booking via bookingId. Data integrity is managed at the API layer via OpenFeign.

## Enumeration Dependency (..>): Domain Constraints

Represents fixed value sets. For example, Booking relies on BookingStatus (BOOKED, CONFIRMED, CANCELLED), and Credential relies on Role (ROLE\_PASSENGER, ROLE\_ADMIN, ROLE\_STAFF).

## 2. Identity & Traveler Domain Model

auth-service (Port 8082) and user-service (Port 8084)

### Credential (Entity) | auth-service | Table: credentials

Primary Key: credentialId : Long | Unique Key: email : String

- \* Purpose: Represents user authentication accounts. Completely decoupled from personal demographic details for maximum security.
- \* Attributes:
  - credentialId : Long [PK] -- Unique auto-increment identifier.
  - userId : Long -- Logical cross-service link pointing to the user profile in user-service.
  - email : String -- Unique user login identifier.
  - password : String -- Irreversible BCrypt cryptographic password hash (60-char string, 12 salt rounds).
  - role : Role -- Assigned security authority (ROLE\_PASSENGER, ROLE\_ADMIN, ROLE\_STAFF).
  - createdAt / updatedAt : LocalDateTime -- Audit lifecycle timestamps.
- \* Methods: verifyPassword(rawPassword : String) : Boolean, updatePassword(newHash : String) : void, getRole() : Role.

### RefreshToken (Entity) & Role (Enum) | auth-service

Relationship: Composition Credential 1 \*- 0..\* RefreshToken

- \* RefreshToken Attributes: id : Long [PK], token : String (UUID v4), expiryDate : Instant, credential : Credential [ManyToOne].
- \* Role Enum Values: ROLE\_PASSENGER (standard traveler purchasing tickets), ROLE\_ADMIN (fleet and flight operational administrator), ROLE\_STAFF (airport gate boarding personnel).
- \* Session Lifecycle: When a user authenticates via POST /api/auth/login, auth-service issues a short-lived JWT (24h) and persists a RefreshToken. Upon token refresh, the old token is rotated and replaced.

### UserEntity (Entity) | user-service | Table: users

Primary Key: userId : Long | Logical 1:1 with Credential (userId)

- \* Purpose: Houses traveler profile demographics and sensitive identity documents, safely decoupled from login credentials.
- \* Attributes:
  - userId : Long [PK] -- Auto-increment primary key.
  - firstName : String (max 50) & lastName : String (max 50) -- Legal travel identity names.
  - email : String -- Contact email address (mirrored from auth-service).
  - phone : String -- Mobile phone number for SMS flight notifications.
  - address : String -- Residential billing address.
  - passportNumber : String -- International travel document number for immigration checks.
  - dateOfBirth : LocalDate -- Passenger age computation.
- \* Methods: getFullName() : String, updateProfile(dto : UserRequest) : void, validatePassport() : Boolean.

### DESIGN DECISION: Why Separate Credential from UserEntity?

By separating Credential (auth-service) from UserEntity (user-service), AeroBook enforces the Principle of Least Privilege. If the profile database is breached, attacker access does NOT yield password hashes or session tokens. Furthermore, GDPR "Right to be Forgotten" profile deletions can be performed without breaking authentication event logs.

### 3. Flight Scheduling & Yield Pricing Domain Model

flight-service (Port 8087) and fare-service (Port 8089)

**Aircraft (Entity) | flight-service | Table: aircrafts**

**Primary Key:** id : Long | **Aggregation:** Aircraft 1 o-- 0..\* Flight

- \* Purpose: Represents physical aircraft fleet assets operated by the airline.
- \* Attributes:
  - id : Long [PK] -- Unique aircraft asset identifier.
  - model : String -- Commercial manufacturer model (e.g. "Boeing 737-800", "Airbus A320neo").
  - capacity : Integer -- Total certified passenger seating capacity.
  - totalRows : Integer & seatsPerRow : Integer -- Physical layout matrix (e.g., 30 rows, 6 seats per row = 180 seats).
- \* Methods: calculateTotalCapacity() : Integer, getSeatMap() : Map<String, String>.

**Flight (Entity) | flight-service | Table: flights**

**Primary Key:** id : Long | **Unique Key:** flightNumber : String (e.g. AI-101)

- \* Purpose: Represents scheduled commercial flight routes across city pairs.
- \* Attributes:
  - id : Long [PK] -- Primary database key.
  - flightNumber : String -- Unique commercial flight code.
  - airlineName : String -- Operating airline carrier name.
  - source : String & destination : String -- IATA airport origin and destination codes (e.g., JFK -> LHR).
  - departureTime : LocalDateTime & arrivalTime : LocalDateTime -- Flight itinerary timestamps.
  - availableSeats : Integer & totalSeats : Integer -- Real-time seat inventory counters.
  - basePrice : BigDecimal & status : FlightStatus (SCHEDULED, DELAYED, BOARDING, DEPARTED, CANCELLED).
- \* Methods: lockSeat() : Boolean, releaseSeat() : void, hasAvailability() : Boolean.

**Fare (Entity) | fare-service | Table: fares**

**Primary Key:** id : Long | **Logical 1:1 with Flight (flightId)**

- \* Purpose: Persists pricing tier rules and dynamic surge multipliers for each scheduled flight.
- \* Attributes:
  - id : Long [PK] -- Auto-increment identifier.
  - flightId : Long -- Foreign flight reference.
  - economyFare : BigDecimal -- Base fare for standard economy seats.
  - businessFare : BigDecimal -- Premium business cabin fare (1.8x base).
  - firstClassFare : BigDecimal -- Luxury first class cabin fare (2.5x base).
  - taxPercentage : BigDecimal & discountPercentage : BigDecimal -- Government taxes and active promo codes.
  - currency : String -- ISO 4217 currency code (e.g. INR, USD).
- \* Methods: calculateTotalFare(cabinClass : String) : BigDecimal, applyDiscount(code : String) : BigDecimal, applySurge(multiplier : BigDecimal) : void.

## 4. Booking Orchestration & Airport Check-In Domain Model

booking-service (Port 8088) and check-in-service (Port 8090)

### Booking (Entity) | booking-service | Table: bookings

**Primary Key:** bookingId : Long | **Unique Key:** pnr : String (6-Char Alphanumeric)

- \* Purpose: Root aggregate entity representing a confirmed flight reservation for one or more travelers.
- \* Attributes:
  - bookingId : Long [PK] -- Auto-increment identifier.
  - pnr : String -- Unique 6-character alphanumeric Passenger Name Record (e.g. "AB7K9Q").
  - userId : Long -- Identifier of the traveler account that purchased the reservation.
  - flightId : Long -- Identifier of the scheduled flight in flight-service.
  - bookingDate : LocalDateTime -- Timestamp when reservation was created.
  - totalFare : BigDecimal -- Grand total amount charged to the passenger.
  - status : BookingStatus -- Operational status enum (BOOKED, CONFIRMED, CANCELLED).
  - passengers : List<Passenger> -- Child collection of passengers traveling under this PNR.
- \* Methods: addPassenger(p : Passenger) : void, confirmReservation() : void, cancelReservation() : void.

### Passenger (Entity) | booking-service | Table: passengers

**Composition:** Booking 1 \*-- 1..\* Passenger (Cascade ALL)

- \* Purpose: Represents an individual traveler flying under a reservation.
- \* Attributes:
  - passengerId : Long [PK] -- Auto-increment passenger identifier.
  - firstName : String & lastName : String -- Passenger legal travel name.
  - age : Integer & gender : String -- Demographics for seating policies.
  - seatNumber : String -- Assigned physical seat coordinate (e.g. "12A").
  - booking : Booking [ManyToOne] -- Owning parent reservation entity.
- \* Methods: assignSeat(seatNumber : String) : void, getPassengerInfo() : String.

### CheckIn (Entity) | check-in-service | Table: check\_in

**Primary Key:** id : Long | **Logical 1:1 with Booking** (bookingId)

- \* Purpose: Manages airport check-in confirmations, digital boarding passes, and departure gate verification.
- \* Attributes:
  - id : Long [PK] -- Unique check-in record identifier.
  - bookingId : Long -- Identifier of the associated booking reservation.
  - passengerName : String -- Confirmed traveler name.
  - seatNumber : String -- Chosen physical seat (Window, Aisle, Middle).
  - boardingPassNumber : String -- Formatted boarding reference (e.g. "BP-1001-12A").
  - checkInTime : LocalDateTime & gate : String -- Check-in timestamp and departure gate.
  - status : String -- Operational status ("CHECKED\_IN", "BOARDED").
- \* Methods: assignSeat(seat : String) : void, generateBoardingPassNumber() : String, markBoarded() : void.

## 5. Cross-Cutting DTOs & Microservice Bounded Contexts

Value Objects, Utilities, and Global Exception Architecture

### PnrGenerator (Utility) | booking-service

#### Algorithm: Cryptographically Secure Base-32 Generation

- \* Architectural Responsibility: Generates an immutable, collision-resistant 6-character alphanumeric Passenger Name Record (PNR).
- \* Ambiguity Elimination: Excludes confusing characters (l, O, 0, 1) from the character pool to prevent airport boarding scanning errors.
- \* Entropy: Uses java.security.SecureRandom with  $32^6$  (1,073,741,824) potential combinations, ensuring zero PNR collisions during peak seasons.
- \* Static Invocation: `String pnr = PnrGenerator.generatePnr();`

### ErrorResponse (DTO) | Cross-Cutting Domain Model

#### Standardized RFC-7807 Compliant Error Payload

- \* Architectural Responsibility: Guarantees uniform error representations across all 7 microservices.
- \* Attributes:
  - timestamp : LocalDateTime -- Exact timestamp when exception occurred.
  - status : Integer -- Standard HTTP status code (400, 401, 404, 409, 500).
  - error : String -- HTTP error phrase (e.g. "Not Found", "Conflict").
  - message : String -- Human-readable, diagnostic explanation of the failure.
- \* Global Translation: Generated by centralized @RestControllerAdvice exception handlers.

### GatewayDocsController | api-gateway (Port 8083)

#### Swagger UI Multi-Service Definition Provider

- \* Architectural Responsibility: Dynamic OpenAPI 3 definition server powering the Swagger UI dropdown selector.
- \* Methods:
  - getServiceOpenApi(@PathVariable serviceId : String) : ResponseEntity<String>
  - getServiceOpenApiModel(serviceId : String) : OpenAPI
- \* Clean Serialization: Uses io.swagger.v3.core.util.Json.mapper() with strict JsonInclude.Include.NON\_NULL settings, preventing empty "\$ref": null properties and resolving Swagger UI resolver crashes platform-wide.

### DOMAIN-DRIVEN DESIGN (DDD): Bounded Contexts in AeroBook

In Domain-Driven Design, a single concept (e.g. "Passenger") can mean different things in different domains:

- \* In user-service, a "User" is an account owner with a home address, phone, and billing coordinates.
- \* In booking-service, a "Passenger" is a ticketed traveler occupying a seat on a specific flight itinerary.
- \* In check-in-service, a "CheckIn" is an airport passenger holding a verified boarding pass at departure gate 12.

Instead of creating one bloated "God Object" with 50 fields, AeroBook separates these into clean, bounded entities.

## 6. Entity Traceability Matrix & Technical Defense FAQ

Class-to-Table Reference & Viva Examination Guide

| Class Name   | Owning Microservice     | Database Table | Primary Key         | Key Relationships                              |
|--------------|-------------------------|----------------|---------------------|------------------------------------------------|
| Credential   | auth-service (8082)     | credentials    | credentialId : Long | 1 *-- 0..* RefreshToken<br>Child of Credential |
| RefreshToken | auth-service (8082)     | refresh_tokens | id : Long           |                                                |
| UserEntity   | user-service (8084)     | users          | userId : Long       | Logical 1:1 with Credential                    |
| Aircraft     | flight-service (8087)   | aircrafts      | id : Long           | 1 o-- 0..* Flight                              |
| Flight       | flight-service (8087)   | flights        | id : Long           | Aggregation child of Aircraft                  |
| Fare         | fare-service (8089)     | fares          | id : Long           | Logical 1:1 with Flight                        |
| Booking      | booking-service (8088)  | bookings       | bookingId : Long    | 1 *-- 1..* Passenger                           |
| Passenger    | booking-service (8088)  | passengers     | passengerId : Long  | Child of Booking                               |
| CheckIn      | check-in-service (8090) | check_in       | id : Long           | Logical 1:1 with Booking                       |

### Class Model Defense & Technical Review FAQ

Essential Answers for Examiners and Evaluators

**Q1: Why is Booking to Passenger modeled as a Composition instead of Aggregation?**

Composition indicates non-shareable, existential ownership. A Passenger item under a booking represents a travel ticket segment. If the parent Booking is deleted, the passengers cannot legally remain in the system. JPA enforces this via CascadeType.ALL and orphanRemoval = true.

**Q2: How do you handle foreign keys when entities are in separate microservices?**

AeroBook avoids cross-database physical foreign keys. Instead, entities store raw Long identifiers (e.g. Booking.flightId). Referential integrity is guaranteed through OpenFeign REST client checks at business transaction boundaries, preventing distributed database locks.

**Q3: Why is BigDecimal used for totalFare and basePrice instead of Double or Float?**

Floating-point types (Double, Float) suffer from IEEE 754 binary rounding errors when executing financial additions and multiplier calculations. BigDecimal guarantees arbitrary-precision arithmetic, ensuring zero financial drift during pricing surge and tax computations.



























